

Integration Manual

for S32 EEP Driver

Document Number: IM11EEPASR4.4 Rev0000R4.0.2 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	4
2.3 About This Manual	4
2.4 Acronyms and Definitions	5
2.5 Reference List	5
3 Building the driver	6
3.1 Build Options	6
3.1.1 GCC Compiler/Assembler/Linker Options	6
3.1.2 GHS Compiler/Assembler/Linker Options	9
3.1.3 DIAB Compiler/Assembler/Linker Options	11
3.2 Files required for compilation	13
3.2.1 Other includes files:	14
3.3 Setting up the plugins	15
4 Function calls to module	17
4.1 Function Calls during Start-up	17
4.2 Function Calls during Shutdown	17
4.3 Function Calls during Wake-up	17
5 Module requirements	18
5.1 Exclusive areas to be defined in BSW scheduler	18
5.1.1 Critical Region Exclusive Matrix	19
5.2 Exclusive areas not available on this platform	20
5.3 Peripheral Hardware Requirements	20
5.4 ISR to configure within AutosarOS - dependencies	20
5.5 ISR Macro	20
5.5.1 Without an Operating System	20
5.5.2 With an Operating System	21
5.6 Other AUTOSAR modules - dependencies	21
5.7 Data Cache Restrictions	22
5.8 User Mode support	22
5.8.1 User Mode support in module	22
5.8.2 User Mode configuration in AutosarOS	22
5.9 Multicore support	23
5.9.1 Multi-Core Capabilities Classification Criteria	23
5.9.2 Multi-Core Type I Capabilities	24
5.9.3 Eep driver multicore usage	26

5.9.4 Assumptions	30
6 Main API Requirements	32
6.1 Main function calls within BSW scheduler	32
6.2 API Requirements	32
6.3 Calls to Notification Functions, Callbacks, Callouts	32
7 Memory allocation	33
7.1 Sections to be defined in Eep_MemMap.h	33
7.2 Linker command file	34
8 Integration Steps	35
9 External assumptions for driver	36



Chapter 1

Revision History

Revision	Date	Author	Description
1.0	30.06.2023	NXP RTD Team	S32 Real-Time Drivers AUTOSAR 4.4 Version 4.0.2 Release

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This integration manual describes the integration requirements for Eep Driver for S32 microcontrollers.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32g274a_bga525
- s32g254a_bga525
- s32g233a_bga525
- s32g234m_bga525
- s32g378a_bga525
- s32g379a_bga525
- s32g398a_bga525
- s32g399a_bga525
- s32g338m_bga525
- s32g339m_bga525
- s32g358a_bga525
- s32g359a_bga525
- s32r45_bga780

All of the above microcontroller devices are collectively named as S32.

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
DET	Default Error Tracer
DEM	Diagnostic Event Manager
ECC	Error Correcting Code
VLE	Variable Length Encoding
N/A	Not Available
MCU	Microcontroller Unit
ECU	Electronic Control Unit
EEPROM	Electrically Erasable Programmable Read-Only Memory
EA	EEPROM Abstraction
USDHC	Ultra Secured Digital Host Controller
EMMC	Enhanced Multi Media Card
EEP	EEPROM driver
XML	Extensible Markup Language

2.5 Reference List

#	Title	Version
1	Specification of Eep Driver	AUTOSAR Release 4.4.0
2	Reference Manual	S32G2 Reference Manual, Rev.7, February 2023
		S32G3 Reference Manual, Rev.3, February 2023
		S32R45 Reference Manual, Rev.3, 12/2021
3	Datasheet	S32G2 Data Sheet, Rev.6, Dec 2022
		S32G3 Data Sheet, Rev.2, RC, Feb 2023
		S32R45 Data Sheet, Rev.3, 11/2022
		VR5510 Data Sheet, Rev.5, April 2022
4	Errata	S32G2: Mask Set Errata for Mask 1P77B v1.0
		S32G2: Mask Set Errata for Mask 0P77B v3.0
		S32G3: Mask Set Errata for Mask 1P72B v2.1
		S32R45, Mask Set Errata for Mask P57D, Rev.2.0

Chapter 3

Building the driver

- [Build Options](#)
- [Files required for compilation](#)
- [Setting up the plugins](#)

This section describes the source files and various compilers, linker options used for building the driver. It also explains the EB Tresos Studio plugin setup procedure.

3.1 Build Options

- [GCC Compiler/Assembler/Linker Options](#)
- [GHS Compiler/Assembler/Linker Options](#)
- [DIAB Compiler/Assembler/Linker Options](#)

The RTD driver files are compiled using:

- NXP GCC 9.2.0 20190812 (Build 1649 Revision gaf57174)
- Green Hills Multi 7.1.6d / Compiler 2020.1.4
- Wind River Diab Compiler 7.0.3

The compiler, assembler, and linker flags used for building the driver are explained below.

The TS_T40D11M40I2R0 part of the plugin name is composed as follows:

- T = Target_Id (e.g. T40 identifies Cortex-M architecture)
- D = Derivative_Id (e.g. D11 identifies S32 platform)
- M = SW_Version_Major and SW_Version_Minor
- I = SW_Version_Patch
- R = Reserved

3.1.1 GCC Compiler/Assembler/Linker Options

3.1.1.1 GCC Compiler Options

Compiler Option	Description
-mcpu=cortex-m7	Targeted ARM processor for which GCC should tune the performance of the code
-mthumb	Generates code that executes in Thumb state
-mlittle-endian	Generate code for a processor running in little-endian mode
-mfpv=fpv5-sp-d16	Specifies the floating-point hardware available on the target
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions
-std=c99	Specifies the ISO C99 base standard
-Os	Optimize for size. Enables all -O2 optimizations except those that often increase code size
-ggdb3	Produce debugging information for use by GDB using the most expressive format available, including GDB extensions if at all possible. Level 3 includes extra information, such as all the macro definitions present in the program
-Wall	Enables all the warnings about constructions that some users consider questionable, and that are easy to avoid (or modify to prevent the warning), even in conjunction with macros
-Wextra	This enables some extra warning flags that are not enabled by -Wall
-pedantic	Issue all the warnings demanded by strict ISO C. Reject all programs that use forbidden extensions. Follows the version of the ISO C standard specified by the aforementioned -std option
-Wstrict-prototypes	Warn if a function is declared or defined without specifying the argument types
-Wundef	Warn if an undefined identifier is evaluated in an #if directive. Such identifiers are replaced with zero
-Wunused	Warn whenever a function, variable, label, value, macro is unused
-Werror=implicit-function-declaration	Make the specified warning into an error. This option throws an error when a function is used before being declared
-Wsign-compare	Warn when a comparison between signed and unsigned values could produce an incorrect result when the signed value is converted to unsigned.
-Wdouble-promotion	Give a warning when a value of type float is implicitly promoted to double
-fno-short-enums	Specifies that the size of an enumeration type is at least 32 bits regardless of the size of the enumerator values.
-funsigned-char	Let the type char be unsigned by default, when the declaration does not use either signed or unsigned
-funsigned-bitfields	Let a bit-field be unsigned by default, when the declaration does not use either signed or unsigned
-fomit-frame-pointer	Omit the frame pointer in functions that don't need one. This avoids the instructions to save, set up and restore the frame pointer; on many targets it also makes an extra register available.

Compiler Option	Description
-fno-common	Makes the compiler place uninitialized global variables in the BSS section of the object file. This inhibits the merging of tentative definitions by the linker so you get a multiple-definition error if the same variable is accidentally defined in more than one compilation unit
-fstack-usage	This option is only used to build test for generation Ram/↔ Stack size report. Makes the compiler output stack usage information for the program, on a per-function basis
-fdump-ipa-all	This option is only used to build test for generation Ram/↔ Stack size report. Enables all inter-procedural analysis dumps
-c	Stop after assembly and produce an object file for each source file
-DS32XX	Predefine S32XX as a macro, with definition 1
-DS32G2XX	Predefine S32G2XX as a macro, with definition 1
-DS32G3XX	Predefine S32G3XX as a macro, with definition 1
-DS32R45	Predefine S32R45 as a macro, with definition 1
-DGCC	Predefine GCC as a macro, with definition 1
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode
-DD_CACHE_ENABLE	Predefine D_CACHE_ENABLE as a macro, with definition 1. Enables data cache initialization in source file system.↔ c under the Platform driver
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initialization in source file system.c under the Platform driver
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initialization in source file system.c under the Platform driver
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPORT↔ RT as a macro, with definition 1. Allows drivers to be configured in user mode.

3.1.1.2 GCC Assembler Options

Assembler Option	Description
-X assembler-with-cpp	Specifies the language for the following input files (rather than letting the compiler choose a default based on the file name suffix)
-mcpu=cortexm7	Targeted ARM processor for which GCC should tune the performance of the code
-mfpu=fpv5-sp-d16	Specifies the floating-point hardware available on the target
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions
-mthumb	Generates code that executes in Thumb state
-c	Stop after assembly and produce an object file for each source file

3.1.1.3 GCC Linker Options

Linker Option	Description
-Wl,-Map,filename	Produces a map file
-T linkerfile	Use linkerfile as the linker script. This script replaces the default linker script (rather than adding to it)
-entry=Reset_Handler	Specifies that the program entry point is Reset_Handler
-nostartfiles	Do not use the standard system startup files when linking
-mcpu=cortexm7	Targeted ARM processor for which GCC should tune the performance of the code
-mthumb	Generates code that executes in Thumb state
-mfpu=fpv5-sp-d16	Specifies the floating-point hardware available on the target
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions
-mlittle-endian	Generate code for a processor running in little-endian mode
-ggdb3	Produce debugging information for use by GDB using the most expressive format available, including GDB extensions if at all possible. Level 3 includes extra information, such as all the macro definitions present in the program
-lc	Link with the C library
-lm	Link with the Math library
-lgcc	Link with the GCC library

3.1.2 GHS Compiler/Assembler/Linker Options

3.1.2.1 GHS Compiler Options

Compiler Option	Description
-cpu=cortexm7	Selects target processor: Arm Cortex M7
-thumb	Selects generating code that executes in Thumb state
-fpu=vfpv5_d16	Specifies hardware floating-point using the v5 version of the VFP instruction set, with 16 double-precision floating-point registers
-fsingle	Use hardware single-precision, software double-precision FP instructions
-C99	Use (strict ISO) C99 standard (without extensions)
-ghstd=last	Use the most recent version of Green Hills Standard mode (which enables warnings and errors that enforce a stricter coding standard than regular C and C++)
-Osize	Optimize for size
-gnu_asm	Enables GNU extended asm syntax support
-dual_debug	Generate DWARF 2.0 debug information
-G	Generate debug information
-keeptempfiles	Prevents the deletion of temporary files after they are used. If an assembly language file is created by the compiler, this option will place it in the current directory instead of the temporary directory

Compiler Option	Description
-Wimplicit-int	Produce warnings if functions are assumed to return int
-Wshadow	Produce warnings if variables are shadowed
-Wtrigraphs	Produce warnings if trigraphs are detected
-Wundef	Produce a warning if undefined identifiers are used in #if preprocessor statements
-unsigned_chars	Let the type char be unsigned, like unsigned char
-unsigned_fields	Bitfields declared with an integer type are unsigned
-no_commons	Allocates uninitialized global variables to a section and initializes them to zero at program startup
-no_exceptions	Disables C++ support for exception handling
-no_slash_comment	C++ style // comments are not accepted and generate errors
-prototype_errors	Controls the treatment of functions referenced or called when no prototype has been provided
-incorrect_pragma_warnings	Controls the treatment of valid #pragma directives that use the wrong syntax
-c	Stop after assembly and produce an object file for each source file
-DS32XX	Predefine S32XX as a macro, with definition 1
-DS32G2XX	Predefine S32G2XX as a macro, with definition 1
-DS32G3XX	Predefine S32G3XX as a macro, with definition 1
-DS32R45	Predefine S32R45 as a macro, with definition 1
-DGHS	Predefine GHS as a macro, with definition 1
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode
-DD_CACHE_ENABLE	Predefine D_CACHE_ENABLE as a macro, with definition 1. Enables data cache initialization in source file system.↵c under the Platform driver
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initialization in source file system.c under the Platform driver
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initialization in source file system.c under the Platform driver
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPORT↵RT as a macro, with definition 1. Allows drivers to be configured in user mode

3.1.2.2 GHS Assembler Options

Assembler Option	Description
-cpu=cortexm7	Selects target processor: Arm Cortex M7
-fpu=vfpv5_d16	Specifies hardware floating-point using the v5 version of the VFP instruction set, with 16 double-precision floating-point registers
-fsingle	Use hardware single-precision, software double-precision FP instructions
-preprocess_assembly_files	Controls whether assembly files with standard extensions such as .s and .asm are preprocessed

Assembler Option	Description
-list	Creates a listing by using the name and directory of the object file with the .lst extension
-c	Stop after assembly and produce an object file for each source file

3.1.2.3 GHS Linker Options

Linker Option	Description
-e Reset_Handler	Make the symbol Reset_Handler be treated as a root symbol and the start label of the application
-T linker_script_file.ld	Use linker_script_file.ld as the linker script. This script replaces the default linker script (rather than adding to it)
-map	Produce a map file
-keepmap	Controls the retention of the map file in the event of a link error
-Mn	Generates a listing of symbols sorted alphabetically/numerically by address
-delete	Instructs the linker to remove functions that are not referenced in the final executable. The linker iterates to find functions that do not have relocations pointing to them and eliminates them
-ignore_debug_references	Ignores relocations from DWARF debug sections when using -delete. DWARF debug information will contain references to deleted functions that may break some third-party debuggers
-Llibrary_path	Points to library_path (the libraries location) for thumb2 to be used for linking
-larch	Link architecture specific library
-lstartup	Link run-time environment startup routines. The source code for the modules in this library is provided in the src/libstartup directory
-lind_sd	Link language-independent library, containing support routines for features such as software floating point, run-time error checking, C99 complex numbers, and some general purpose routines of the ANSI C library
-v	Prints verbose information about the activities of the linker, including the libraries it searches to resolve undefined symbols
-nostartfiles	Controls the start files to be linked into the executable

3.1.3 DIAB Compiler/Assembler/Linker Options

3.1.3.1 DIAB Compiler Options

Compiler Option	Description
-tARMCORTEXM7MG:simple	Selects target processor (hardware single-precision, software double-precision floating-point)
-mthumb	Selects generating code that executes in Thumb state
-std=c99	Follows the C99 standard for C
-Oz	Like -O2 with further optimizations to reduce code size
-g	Generates DWARF 4.0 debug information
-fstandalone-debug	Emits full debug info for all types used by the program

Compiler Option	Description
-Wstrict-prototypes	Warn if a function is declared or defined without specifying the argument types
-Wsign-compare	Produce warnings when comparing signed type with unsigned type
-Wdouble-promotion	Give a warning when a value of type float is implicitly promoted to double
-Wunknown-pragmas	Issues a warning for unknown pragmas
-Wundef	Warns if an undefined identifier is evaluated in an #if directive. Such identifiers are replaced with zero
-Wextra	Enables some extra warning flags that are not enabled by '-Wall'
-Wall	Enables all of the most useful warnings (for historical reasons this option does not literally enable all warnings)
-pedantic	Emits a warning whenever the standard specified by the -std option requires a diagnostic
-Werror=implicit-function-declaration	Generates an error whenever a function is used before being declared
-fno-common	Compile common globals like normal definitions
-fno-signed-char	Char is unsigned
-fno-trigraphs	Do not process trigraph sequences
-V	Displays the current version number of the tool suite
-c	Stop after assembly and produce an object file for each source file
-DS32XX	Predefine S32XX as a macro, with definition 1
-DS32G2XX	Predefine S32G2XX as a macro, with definition 1
-DS32G3XX	Predefine S32G3XX as a macro, with definition 1
-DS32R45	Predefine S32R45 as a macro, with definition 1
-DDIAB	Predefine DIAB as a macro, with definition 1
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode
-DD_CACHE_ENABLE	Predefine D_CACHE_ENABLE as a macro, with definition 1. Enables data cache initialization in source file system.c under the Platform driver
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initialization in source file system.c under the Platform driver
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initialization in source file system.c under the Platform driver
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPORT as a macro, with definition 1. Allows drivers to be configured in user mode

3.1.3.2 DIAB Assembler Options

Assembler Option	Description
-mthumb	Selects generating code that executes in Thumb state
-Xpreprocess-assembly	Invokes C preprocessor on assembly files before running the assembler
-Xassembly-listing	Produces an .lst assembly listing file
-c	Stop after assembly and produce an object file for each source file

3.1.3.3 DIAB Linker Options

Linker Option	Description
-e Reset_Handler	Make the symbol Reset_Handler be treated as a root symbol and the start label of the application
linker_script_file.dld	Use linker_script_file.dld as the linker script. This script replaces the default linker script (rather than adding to it)
-m30	m2 + m4 + m8 + m16
-Xstack-usage	Gathers and display stack usage at link time
-Xpreprocess-lecl	Perform pre-processing on linker scripts
-Llibrary_path	Points to the libraries location for ARMV7EMMG to be used for linking
-lc	Links with the standard C library
-lm	Links with the math library

3.2 Files required for compilation

This section describes the include files required to compile, assemble (if assembler code) and link the Eep driver for S32 microcontrollers. To avoid integration of incompatible files, all the include files from other modules shall have the same AR_MAJOR_VERSION and AR_MINOR_VERSION, i.e. only files with the same AUTOSAR major and minor versions can be compiled.

#Eep files:

- Eep_TS_T40D11M40I2R0\include\Eep.h
- Eep_TS_T40D11M40I2R0\include\Eep_InternalTypes.h
- Eep_TS_T40D11M40I2R0\include\Eep_IPW.h
- Eep_TS_T40D11M40I2R0\include\Eep_Types.h
- Eep_TS_T40D11M40I2R0\include\Sd_Emmc_Ip.h
- Eep_TS_T40D11M40I2R0\include\Sd_Emmc_Ip_Types.h
- Eep_TS_T40D11M40I2R0\include\Usdhc_Ip.h
- Eep_TS_T40D11M40I2R0\include\Usdhc_Ip_Types.h
- Eep_TS_T40D11M40I2R0\src\Eep.c
- Eep_TS_T40D11M40I2R0\src\Eep_IPW.c
- Eep_TS_T40D11M40I2R0\src\Sd_Emmc_Ip.c
- Eep_TS_T40D11M40I2R0\src\Usdhc_Ip.c

Note

- Eep_PBcfg.c - this file should be generated by the user using a configuration/generation tool
- Eep_PBcfg.h - this file should be generated by the user using a configuration/generation tool
- Eep_Cfg.h - this file should be generated by the user using a configuration/generation tool
- Eep_Cfg.c - this file should be generated by the user using a configuration/generation tool
- Sd_Emmc_Ip_PBcfg.c - this file should be generated by the user using a configuration/generation tool
- Sd_Emmc_Ip_Cfg.h - this file should be generated by the user using a configuration/generation tool
- Usdhc_Ip_PBcfg.c - this file should be generated by the user using a configuration/generation tool
- Usdhc_Ip_Cfg.h - this file should be generated by the user using a configuration/generation tool
- Usdhc_Ip_DeviceRegisters.h - this file should be generated by the user using a configuration/generation tool

3.2.1 Other includes files:

3.2.1.1 Files from MemIf folder:

- MemIf_TS_T40D11M40I2R0\include\MemIf_Types.h

3.2.1.2 Files from Base common folder:

- BaseNXP_TS_T40D11M40I2R0\include\Compiler.h
- BaseNXP_TS_T40D11M40I2R0\include\Compiler_Cfg.h
- BaseNXP_TS_T40D11M40I2R0\include\ComStack_Types.h
- BaseNXP_TS_T40D11M40I2R0\include\Fls_MemMap.h
- BaseNXP_TS_T40D11M40I2R0\include\Mcal.h
- BaseNXP_TS_T40D11M40I2R0\include\OsIf.h
- BaseNXP_TS_T40D11M40I2R0\include\Platform_Types.h
- BaseNXP_TS_T40D11M40I2R0\include\Std_Types.h
- BaseNXP_TS_T40D11M40I2R0\include\Reg_eSys.h
- BaseNXP_TS_T40D11M40I2R0\include\Soc_Ips.h
- BaseNXP_TS_T40D11M40I2R0\include\Reg_Macros.h
- BaseNXP_TS_T40D11M40I2R0\include\SilRegMacros.h
- BaseNXP_TS_T40D11M40I2R0\header\S32G274A.h

3.2.1.3 Files from Det folder:

- Det_TS_T40D11M40I2R0\include\Det.h

3.2.1.4 Files from Dem folder:

- Dem_TS_T40D11M40I2R0\include\Dem.h

3.2.1.5 Files from Mcl folder:

- Mcl_TS_T40D11M40I2R0\include\Cache_Ip.h

3.2.1.6 Files from Platform folder:

- Platform_TS_T40D11M40I2R0\include\Platform.h

3.2.1.7 Files from Rte folder:

- Rte_TS_T40D11M40I2R0\include\SchM_Eep.h

3.2.1.8 Files from Os folder:

- Os_TS_T40D11M40I2R0\include\Os.h

3.2.1.9 Files from Rm folder:

- Rm_TS_T40D11M40I2R0\include\CDD_Rm.h
- Rm_TS_T40D11M40I2R0\include\Sema42_Ip.h
- Rm_TS_T40D11M40I2R0\include\Xrdc_Ip.h

3.3 Setting up the plugins

The Eep Driver was designed to be configured by using the EB Tresos Studio (version 27.1.0 or later)

3.3.0.0.1 Steps to generate the configuration:

1. Copy the module folders:
 - BaseNXP_TS_T40D11M40I2R0
 - Det_TS_T40D11M40I2R0
 - Dem_TS_T40D11M40I2R0
 - Mcl_TS_T40D11M40I2R0
 - EcuC_TS_T40D11M40I2R0
 - Eep_TS_T40D11M40I2R0
 - Rm_TS_T40D11M40I2R0
 - MemIf_TS_T40D11M40I2R0
 - Os_TS_T40D11M40I2R0
 - Platform_TS_T40D11M40I2R0
 - Resource_TS_T40D11M40I2R0
 - Rte_TS_T40D11M40I2R0 into the Tresos plugins folder.
2. Set the desired Tresos Output location folder for the generated sources and header files.
3. Use the EB tresos Studio GUI to modify ECU configuration parameters values.
4. Generate the configuration files.

3.3.0.0.2 Location of various files inside the EEP module folder:

- VSMD (Vendor Specific Module Definition) file in EB Tresos Studio XDM format:
 - Eep_TS_T40D11M40I2R0\config\Eep.xdm
- VSMD (Vendor Specific Module Definition) file(s) in AUTOSAR compliant EPD format:
 - Eep_TS_T40D11M40I2R0\autosar\Eep_<subderivative_name>.epd
- Code Generation Templates for variant aware parameters:
 - Eep_TS_T40D11M40I2R0\generate_PB\include\Eep_PBcfg.h
 - Eep_TS_T40D11M40I2R0\generate_PB\src\Eep_PBcfg.c
 - Eep_TS_T40D11M40I2R0\generate_PB\src\Sd_Emmc_Ip_PBcfg.c
 - Eep_TS_T40D11M40I2R0\generate_PB\src\Usdhc_Ip_PBcfg.c
- Code Generation Templates for parameters without variation points:
 - Eep_TS_T40D11M40I2R0\generate_PC\include\Eep_Cfg.h
 - Eep_TS_T40D11M40I2R0\generate_PC\include\Sd_Emmc_Ip_Cfg.h
 - Eep_TS_T40D11M40I2R0\generate_PC\include\Usdhc_Ip_Cfg.h
 - Eep_TS_T40D11M40I2R0\generate_PC\include\Usdhc_Ip_Features.h
 - Eep_TS_T40D11M40I2R0\generate_PC\src\Eep_Cfg.c



Chapter 4

Function calls to module

- [Function Calls during Start-up](#)
- [Function Calls during Shutdown](#)
- [Function Calls during Wake-up](#)

4.1 Function Calls during Start-up

EEP shall be initialized during STARTUP phase of EcuM initialization. The API to be called for this is Eep_Init(). The MCU module should be initialized before the EEP is initialized.

4.2 Function Calls during Shutdown

None.

4.3 Function Calls during Wake-up

None.

Chapter 5

Module requirements

- Exclusive areas to be defined in BSW scheduler
- Exclusive areas not available on this platform
- Peripheral Hardware Requirements
- ISR to configure within AutosarOS - dependencies
- ISR Macro
- Other AUTOSAR modules - dependencies
- Data Cache Restrictions
- User Mode support
- Multicore support

5.1 Exclusive areas to be defined in BSW scheduler

- In the current implementation, EEP is using the services of Run-Time Environment (RTE) for entering and exiting the critical regions.
- RTE implementation is done by the integrators of the RTD using OS or non-OS services.
- For testing the EEP, stubs are used for RTE.
- EEP driver has four exclusive areas (EA) EEP_EXCLUSIVE_AREA_01, EEP_EXCLUSIVE_AREA_02, EEP_EXCLUSIVE_AREA_03 and EEP_EXCLUSIVE_AREA_04.
- The purpose of these exclusive areas is to make the functions Eep_Erase, Eep_Write, Eep_Read and Eep_Compare thread safe and thus protect EEP internal job variables.

EEP_EXCLUSIVE_AREA_01 is used in function Eep_Erase. It protects the variables:

- Eep_u32EepromAddrIt
- Eep_u32RemainingLength

- Eep_eJob
- Eep_eJobResult

EEP_EXCLUSIVE_AREA_02 is used in function Eep_Write. It protects the variables:

- Eep_u32EepromAddrIt
- Eep_pu8JobSrcAddrPtr
- Eep_u32RemainingLength
- Eep_eJob
- Eep_eJobResult

EEP_EXCLUSIVE_AREA_03 is used in function Eep_Read. It protects the variables:

- Eep_u32EepromAddrIt
- Eep_pu8JobDataDestPtr
- Eep_u32RemainingLength
- Eep_eJob
- Eep_eJobResult

EEP_EXCLUSIVE_AREA_04 is used in function Eep_Compare. It protects the variables:

- Eep_u32EepromAddrIt
- Eep_pu8JobSrcAddrPtr
- Eep_u32RemainingLength
- Eep_eJob
- Eep_eJobResult

5.1.1 Critical Region Exclusive Matrix

- Below is the table depicting the exclusivity between different critical region IDs from the EEP driver.
- If there is an “X” in a table, it means that those 2 critical regions cannot interrupt each other.

Exclusive Area ID	DRIVER_EXCLUSIVE_AREA_01 Eep_Erase	DRIVER_EXCLUSIVE_AREA_02 (Eep_Write)	DRIVER_EXCLUSIVE_AREA_03 (Eep_Read)	DRIVER_EXCLUSIVE_AREA_04 (Eep_Compare)
DRIVER_EXCLUSIVE_AREA_01 Eep_Erase		x	x	x
DRIVER_EXCLUSIVE_AREA_02 (Eep_Write)	x		x	x
DRIVER_EXCLUSIVE_AREA_03 (Eep_Read)	x	x		x
DRIVER_EXCLUSIVE_AREA_04 (Eep_Compare)	x	x	x	

5.2 Exclusive areas not available on this platform

List of exclusive areas which are not available on this platform (or blank if they're all available).

- EEP_EXCLUSIVE_AREA__00
- EEP_EXCLUSIVE_AREA__05
- EEP_EXCLUSIVE_AREA__06
- EEP_EXCLUSIVE_AREA__07
- EEP_EXCLUSIVE_AREA__08
- EEP_EXCLUSIVE_AREA__09
- EEP_EXCLUSIVE_AREA__10
- EEP_EXCLUSIVE_AREA__11
- EEP_EXCLUSIVE_AREA__12
- EEP_EXCLUSIVE_AREA__13
- EEP_EXCLUSIVE_AREA__14

5.3 Peripheral Hardware Requirements

The EEP driver uses/controls the "USDHC" MCU peripheral. For more details about peripheral and its structure refers to MCU reference manual.

5.4 ISR to configure within AutosarOS - dependencies

ISR Name	HW INT Vector	Observations
uSDHC0_Isr	36	None

5.5 ISR Macro

RTD drivers use the ISR macro to define the functions that will process hardware interrupts. Depending on whether the OS is used or not, this macro can have different definitions.

5.5.1 Without an Operating System

The macro *USING_OS_AUTOSAROS* must not be defined.

5.5.1.1 Using Software Vector Mode

The macro `USE_SW_VECTOR_MODE` must be defined and the ISR macro is defined as:

```
#define ISR(IsrName) void IsrName(void)
```

In this case, the drivers' interrupt handlers are normal C functions and their prologue/epilogue will handle the context save and restore.

5.5.1.2 Using Hardware Vector Mode

The macro `USE_SW_VECTOR_MODE` must not be defined and the ISR macro is defined as:

```
#define ISR(IsrName) INTERRUPT_FUNC void IsrName(void)
```

In this case, the drivers' interrupt handlers must also handle the context save and restore.

5.5.2 With an Operating System Please refer to your OS documentation for description of the ISR macro.

5.6 Other AUTOSAR modules - dependencies

- **Base:** This module provides basic data types and auxiliary macros or functions commonly used by other AUTOSAR modules.
- **Dem:** This module is necessary for enabling reporting of production relevant error status. The API function used is `Dem_ReportErrorStatus()`.
- **Det** This module is necessary for enabling Development error detection. The API function used is `Det_ReportError()`. The activation/deactivation of Development error detection is configurable using 'EepDevErrorDetect' configuration parameter.
- **EcuC:** (only for ASR 4.2) The ECUC module is used for ECU configuration. RTD modules need ECUC to retrieve the variant information.
- **Rte:** Exclusive areas implementations.
- **MemIf:** Memory Interface.
- **Mcl:** Clean/invalidate cache In the DMA transfer modes.
- **Rm:** Provide Semaphore which is needed in multicore.

5.7 Data Cache Restrictions

In the DMA transfer mode, DMA transfers may issue cache coherency problems. To avoid possible coherency issues when D-CACHE is enabled, the user shall ensure that the buffers used as TCD source and destination are allocated in the NON-CACHEABLE area (by means of `Eep_Memmap`) or enable the feature **EepSynchronizeCache** to clean/invalidate data cache before transferring.

Note

- In case the user's buffer address or length are not aligned with the cache line size (32-byte), driver will decrease the start address and increase the length to compensate, to ensure both of them are aligned
- It is highly recommended to define the DMA buffers aligned to the cache line size boundary, and the buffer length is a multiple of the cache line size to avoid unexpected behavior

5.8 User Mode support

- [User Mode support in module](#)
- [User Mode configuration in AutosarOS](#)

5.8.1 User Mode support in module

No special measures need to be taken to run Eep module from user mode. The Eep driver code can be executed at any time from both supervisor and user mode.

5.8.2 User Mode configuration in AutosarOS

When User mode is enabled, the driver may have the functions that need to be called as trusted functions in AutosarOS context. Those functions are already defined in driver and declared in the header `<IpName>_IpTrustedFunctions.h`. This header also included all headers files that contains all types definition used by parameters or return types of those functions. Refer the chapter [User Mode support in module](#) for more detail about those functions and the name of header files they are declared inside. Those functions will be called indirectly with the naming convention below in order to AutosarOS can call them as trusted functions.

```
Call_<Function_Name>_TRUSTED(parameter1,parameter2,...)
```

That is the result of macro expansion `OsIf_Trusted_Call` in driver code:

```
#define OsIf_Trusted_Call[1-6params](name,param1,...,param6) Call_##name##_TRUSTED(param1,...,param6)
```

So, the following steps need to be done in AutosarOS:

- Ensure `MCAL_ENABLE_USER_MODE_SUPPORT` macro is defined in the build system or somewhere global.
- Define and declare all functions that need to call as trusted functions follow the naming convention above in Integration/User code. They need to be visible in `Os.h` for the driver to call them. They will do the marshalling of the parameters and call `CallTrustedFunction()` in OS specific manner.
- `CallTrustedFunction()` will switch to privileged mode and call `TRUSTED_<Function_Name>()`.

- `TRUSTED_<Function_Name>()` function is also defined and declared in Integration/User code. It will un-marshalling of the parameters to call `<Function_Name>()` of driver. The `<Function_Name>()` functions are already defined in driver and declared in `<IpName>_Ip_TrustedFunctions.h`. This header should be included in OS for OS call and indexing these functions.

See the sequence chart below for an example calling `LinfleXd_Uart_Ip_Init_Privileged()` as a trusted function.

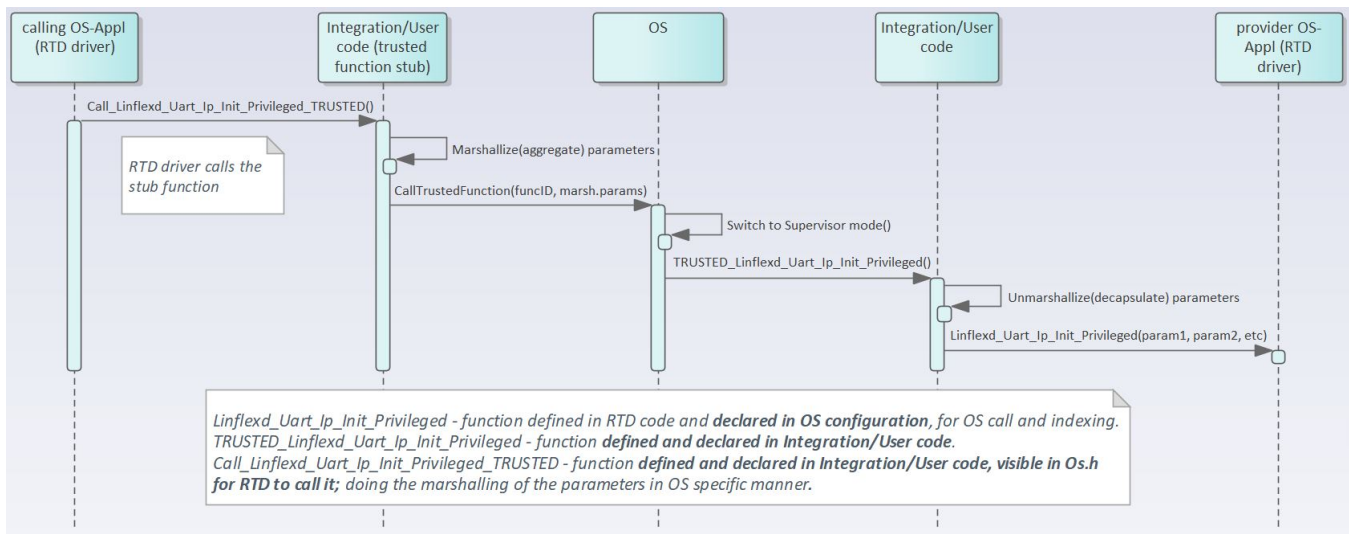


Figure 5.1 Example sequence chart for calling `LinfleXd_Uart_Ip_Init_Privileged` as trusted function

5.9 Multicore support

- The Eep implements the **Autosar 4.4 MCAL Multicore Distribution** according to **type I**, in which the mappable element is set to HW Unit (the uSDHC hardware controller)
- For additional details, please refer to **AUTOSAR_EXP_BSWDistributionGuide**

Driver	HW "Natural" Element	Mappable Element (ME)	Relation (ME:Core)	Multi-Core Type
Eep	EEPROM Driver	MCAL Module	1:1	Type I

Note

- Multicore is only supported in the HLD (in Autosar context)
- The IP layers (SD_eMMC and uSDHC) do not have this feature (when using them only - in Non-Autosar context)

5.9.1 Multi-Core Capabilities Classification Criteria

Module requirements

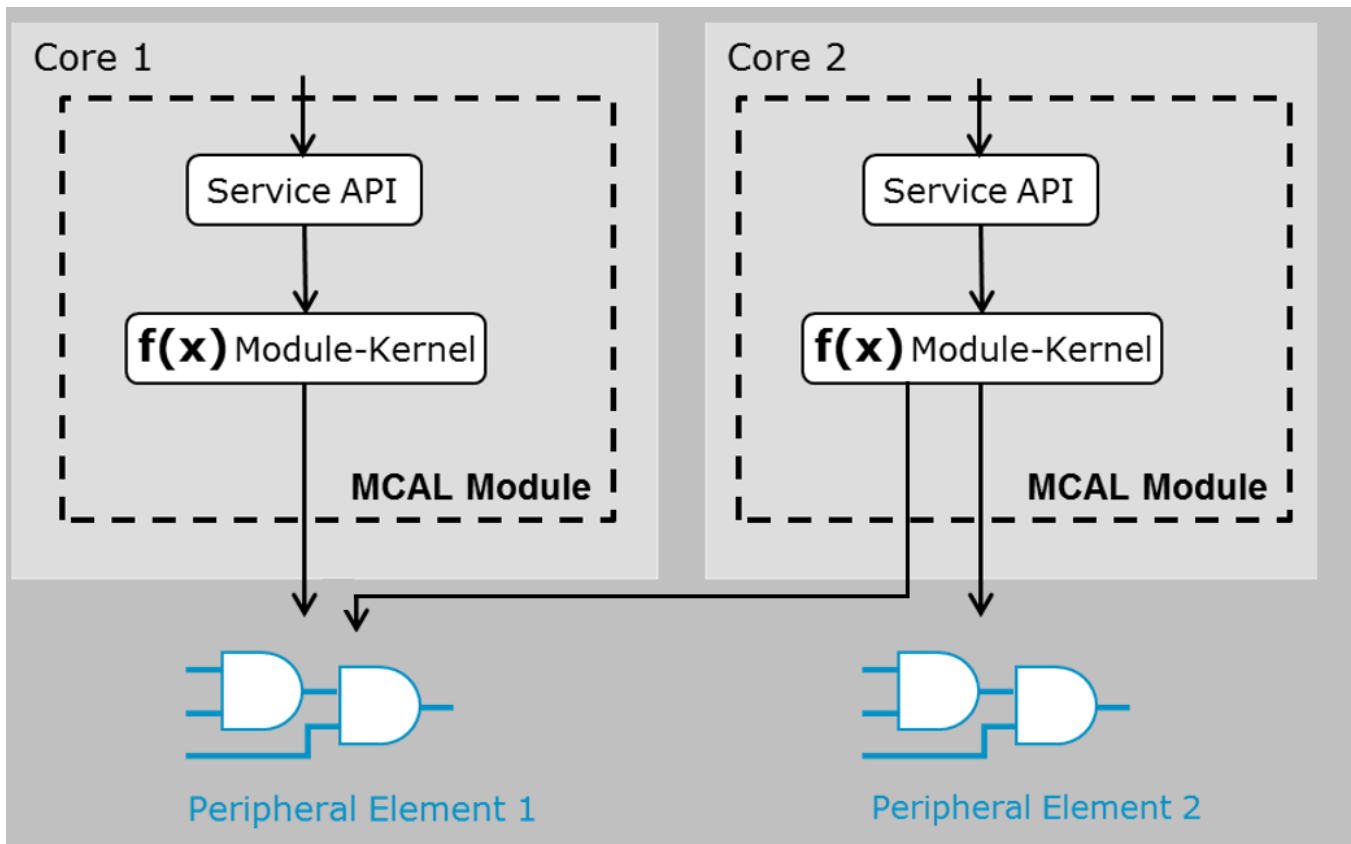
The following paragraphs are given to unify the understanding the required multi-core capabilities from different point of perspective.

- **Criteria 1 - APIs Availability:** To classify the multi-core capability of a MCAL module it is first essential to understand the user expectation in the sense of from what core the service APIs shall be reachable. Out of this definition the following two cases can be derived:
 - **1a:** Local service APIs (executable on one core only)
 - **1b:** Global (distributed/shared) service APIs (executable on any core)
- **Criteria 2 MCAL Kernel Execution Context:** Secondly one needs to understand where the MCAL module kernel shall ideally reside/located to limit the side effects of collisions on busses and bridges due to concurrent access to HW peripherals from several cores. Defining a local kernel does not exclude a multiplicity, e.g. to provide several kernels dealing with independent peripheral modules or core individual resources. The following cases are defined:
 - **2a:** One Local kernel (executable on one core only - separated binary file for each core)
 - **2b:** Global (distributed/shared) kernel (executable on any core - one binary file for all cores)
- **Criteria 3 HW Elements Mapping:** As a third point one needs to consider the scope of mappable elements, including its data to the according kernel instance. Taking this aspect into account, one extends the classification according to the hardware capabilities in terms of mapping of HW peripherals to cores. Here not only the pure hardware capability needs to be considered but also the performance impact of the according mapping. The following cases are defined:
 - **3a:** One HW element mappable to one core only
 - **3b:** One HW element mappable to several cores
- The following table summarizes the scope of the required options for the shown criteria.

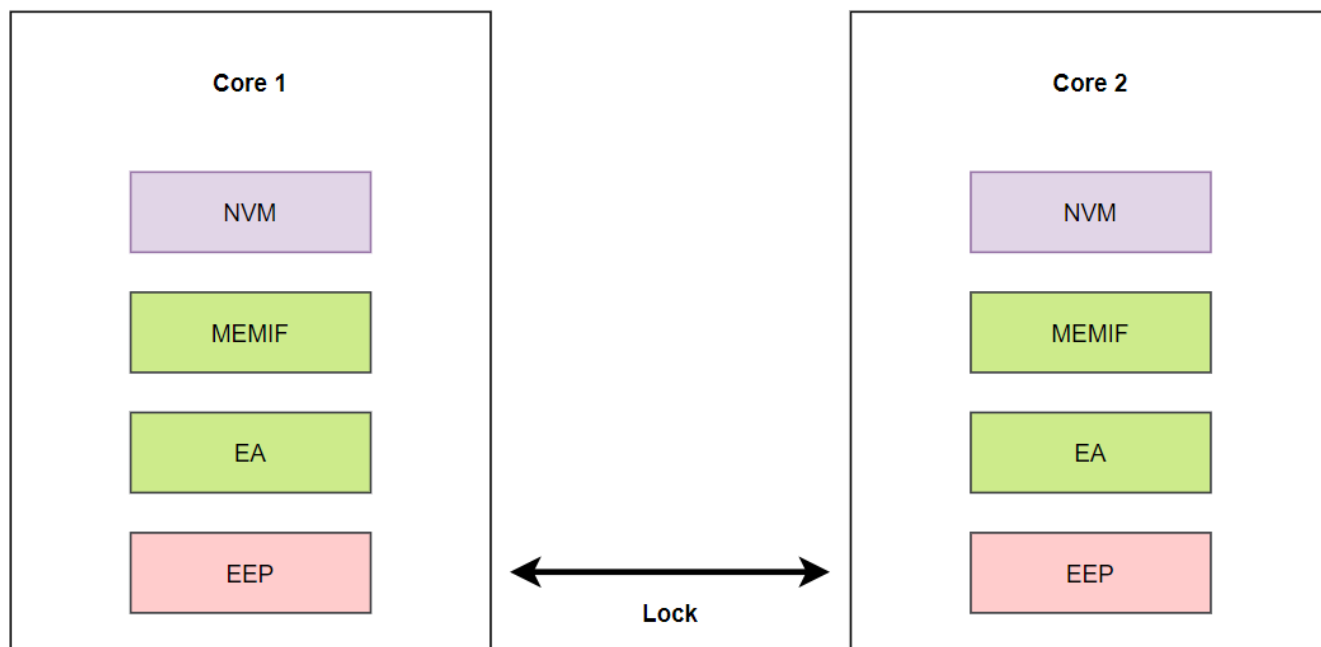
Criteria	One Core Only	Several Cores
APIs	1a	1b
Kernel Execution Context	2a	2b
HW Elements	3a	3b

5.9.2 Multi-Core Type I Capabilities

- **Type I = 1a + 2a + 3a**
 - MCAL modules available on a single core only
 - The interfaces are not globally available



- A single-core module providing its service APIs to one core only and implementing the kernel on exactly this core as the according HW elements shall be accessed by one core only
 - No data exchange between the cores
 - Each core accesses the memory by its own memory stack (kernel) in its binary file
 - Synchronizes memory driver access by a locking mechanism (e.g: Semaphore)



5.9.3 Eep driver multicore usage

- The Eep driver supports multicore synchronization if **EepMCoreEnable** is enabled

Eep

Name

General EepUsdhcControllerCfg EepSdCfg Published Information

AutosarExt

Name

Eep Enable User Mode Support ☐

Eep Multi Cores Enable ☒

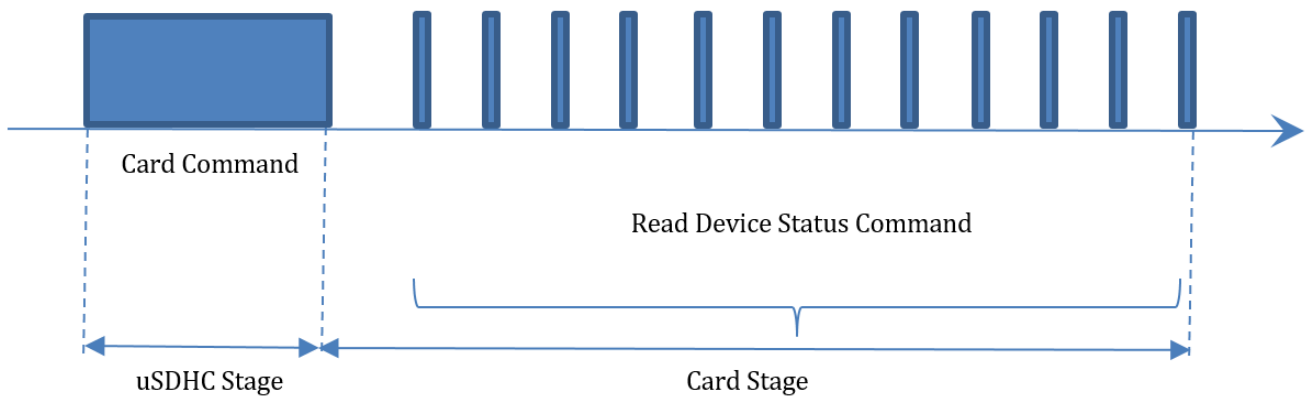
Eep MultiCores Semaphore Channel Number (0x0 -> 0xffffffff)

Eep MCore Lock Gates Timeout (0x0 -> 0xffffffff)

- When the synchronization is enabled, the Multicore related parameters (semaphore channel and timeout value) have to be configured accordingly on both cores
- The synchronization feature arbitrates the access to the resource, using the configured hardware semaphores to ensure there is no access conflict from two different cores on the same hardware registers
 - The core which wins the arbitration will access the resource and start the job in hardware
 - The core which loses the arbitration will mark it's current job as pending until the controller is idle or until timeout occurs

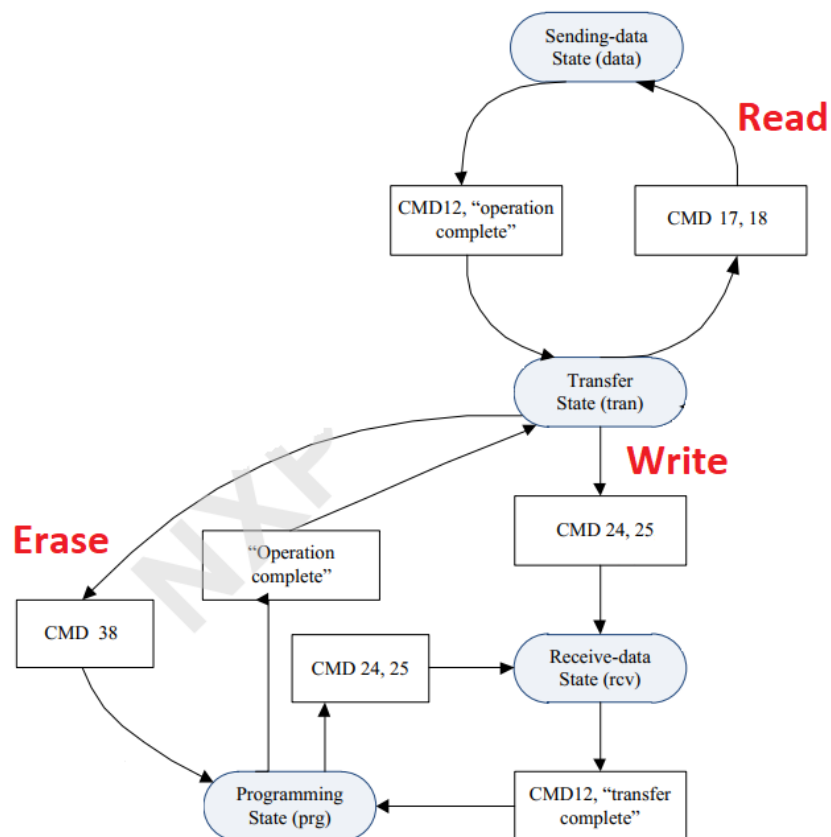
5.9.3.1 Card operation

- The memory card operations consist of two stages:
 - In stage 1 (uSDHC stage) the command is sent to the memory card via uSDHC controller
 - In stage 2 (Card Stage) the actual operation takes place in the memory card and the MCU must poll the status of the operation to find when it ends



Note

- Card stage on Write and Erase operations: Programming State (prg)
- Card stage on Read operation: Sending-data State (data)



Module requirements

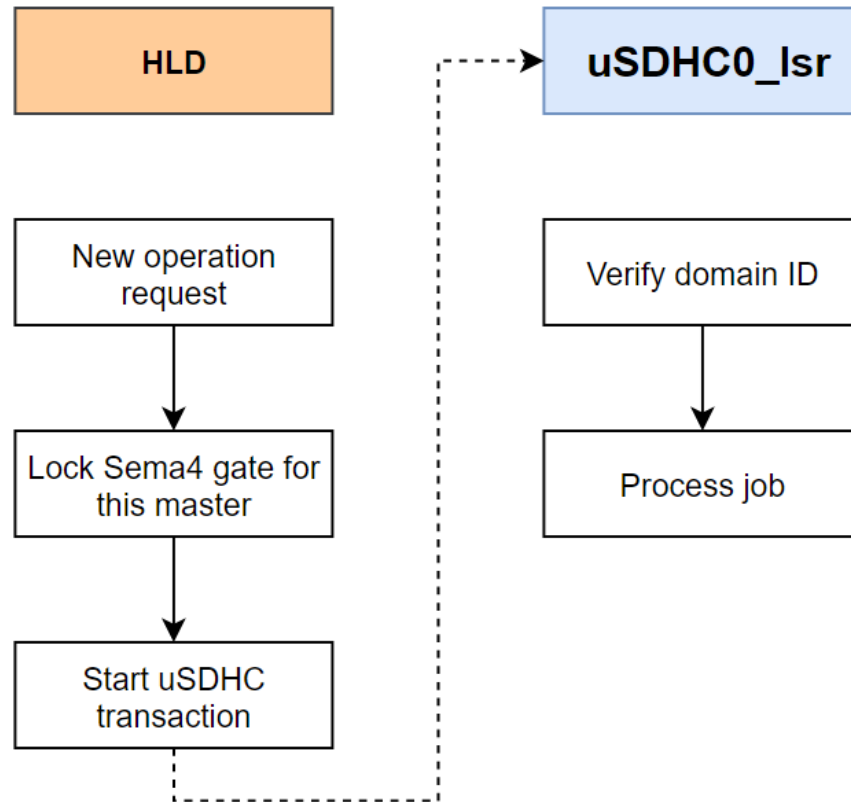
- In multicore context, the general restriction is that no two operations (read, write, erase) may be performed in parallel
- The uSDHC stages of different operations must not overlap because the uSDHC module can only do one transmission at a time
- This restriction includes the card stage too, following the device state transition in data transfer mode from SD/eMMC specifications:
 - Read commands are not allowed while Device is in **Programming** state
 - Write and Erase commands are not allowed while Device is in **Sending-data** state
- So both stages depicted above must be performed as one atomic operation (not interruptible by other cores)

5.9.3.2 Semaphore locking

- Eep driver uses the spinlock mechanism to allow accessing the same hardware element from different cores.
- uSDHC / memory card can be protected against concurrent accesses by a hardware semaphore. Each core that wants to make any memory operations (read, write or erase) through uSDHC must first take the semaphore. This is done internally by the Eep driver. The semaphore is released at the end of the operation
- To further guard against interference, XRDC can be set to allow write access to uSDHC registers only if the semaphore is owned by the respective domain. This ensures the current memory operation cannot be impacted by writes to uSDHC registers performed by another (faulty) core
- For all operations (read, write and erase) the semaphore must be held for the entire duration of the operation, including the card stage. Starting another operation during the card stage would result in the command being ignored by the device
- For large operations, which are internally split by the driver in several smaller operations, the semaphore is held for the duration of the entire operation
- The semaphore is taken on first come first served basis, there is no arbitration between cores. This can potentially lead to denial of service for an indeterminate amount of time for a core, if other cores continuously perform card operations and grab the semaphore first

5.9.3.3 Interrupt handling

- In multicore context, the uSDHC interrupt is shared between cores, it means an event will trigger all cores to execute the ISR handlers
- The interrupt handler will only serve the master domain that owns the semaphore (the semaphore was locked at the beginning of the operation)



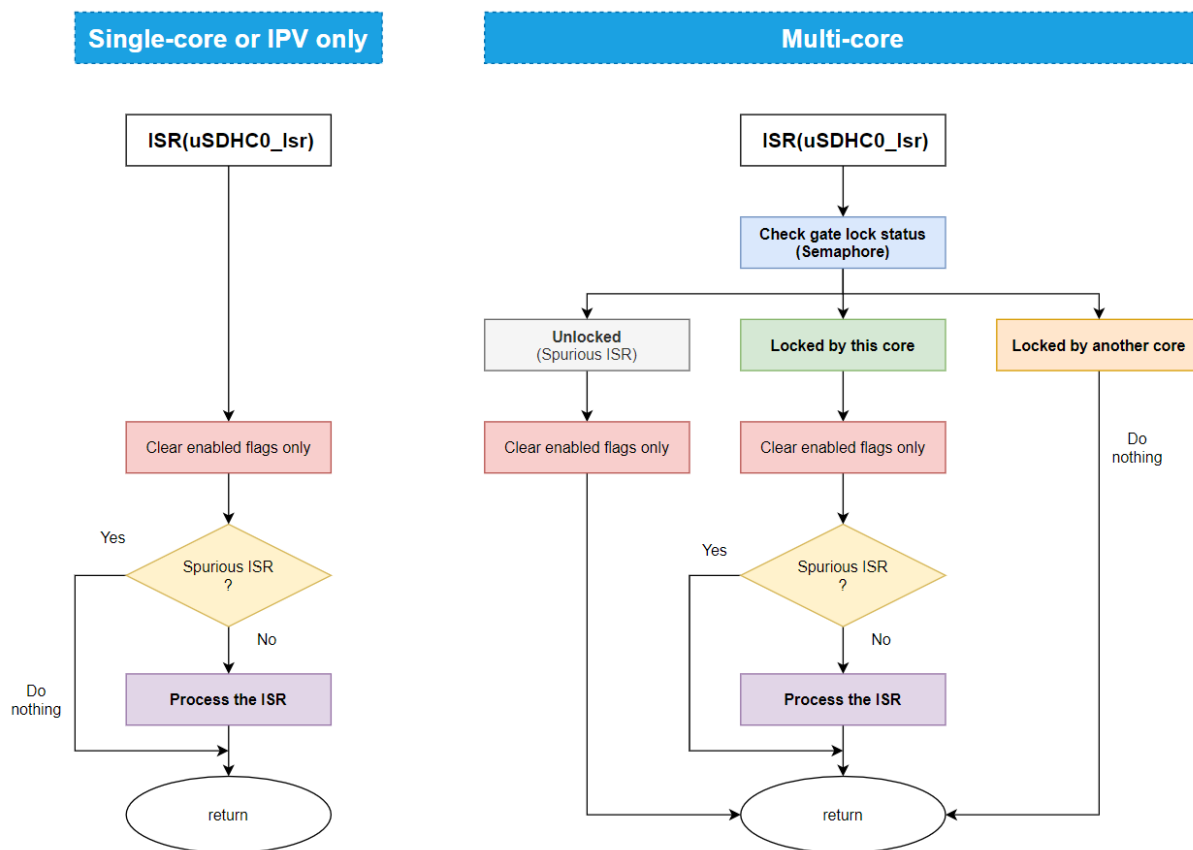
- The process will be as follows:

- HLD

- * A new operation requested, Eep driver locks the semaphore
- * After that, a new uSDHC transaction starts in the IP layer

- ISR

- * Checks the semaphore status to identify which domain is locking it
- * Only and only if the locker belongs to this domain, the handler processes the ISR, ignore otherwise



5.9.3.4 Error recovery

- If one of the cores crashes while owning the semaphores it is possible to recover the semaphore by forcefully resetting it from outside the Eep driver. This mechanism allows either individual or all gates to be reset by a separate domain
- If the application has no other way of knowing that another core has crashed, a timeout can be used. The timeout value depends on the application (one core may experience delays caused by multiple operations started from the other cores, as there is no arbitration when taking the semaphore)
- After the semaphore reset any core can initiate card operations

5.9.4 Assumptions

The Eep driver and the mappable element can be allocated to one or several cores. If the Eep driver is used in multi-core context, it enforces the following assumptions:

1. Each core will have a separated binary files
2. The Eep driver assumes that initialization is performed on each core, `Eep_Init()` is called separately for each core
 - Users are responsible for ensuring configuration synchronization between cores

- The configuration of one core should be compatible with the other cores
 - For instance, if two cores work with two memory cards through a uSDHC hardware instance, it is impossible when a core work in 3V3 mode and the other core works in 1V8 mode. Or different data bus width modes
3. All cores must use the same hardware semaphore channel number
 4. The uSDHC ISR must be enabled on all cores to serve the driver operations of that core

Chapter 6

Main API Requirements

- [Main function calls within BSW scheduler](#)
- [API Requirements](#)
- [Calls to Notification Functions, Callbacks, Callouts](#)

6.1 Main function calls within BSW scheduler

Eep_MainFunction (call rate depends on target application, i.e. how fast the data needs to be read/written/compared into SD/EMMC memory)

6.2 API Requirements

None.

6.3 Calls to Notification Functions, Callbacks, Callouts

The EEP driver provides notifications that are user configurable:

- EepJobEndNotification (usually routed to EA module)
- EepJobErrorNotification (usually routed to EA module)

Chapter 7

Memory allocation

- [Sections to be defined in Eep_MemMap.h](#)
- [Linker command file](#)

7.1 Sections to be defined in Eep_MemMap.h

Section name	Type of section	Description
EEP_START_SEC_CONFIG_DATA_32	Configuration Data	Used for 32 bit Configuration
EEP_STOP_SEC_CONFIG_DATA_32	Configuration Data	End of above section
EEP_START_SEC_CONFIG_DATA_UNSPECIFIED	Configuration Data	Used for Configuration when the SIZE (alignment) does not fit the criteria of 8,16 or 32 bit
EEP_STOP_SEC_CONFIG_DATA_UNSPECIFIED	Configuration Data	End of above section
EEP_START_SEC_CODE	Code	Start of memory Section for Code
EEP_STOP_SEC_CODE	Code	End of above section
EEP_START_SEC_CONST_UNSPECIFIED	Constant Data	The parameters that are not variant aware shall be stored in memory section for constants.
EEP_STOP_SEC_CONST_UNSPECIFIED	Constant Data	End of above section
EEP_START_SEC_VAR_INIT_BOOLEAN	Variables	Used for boolean variables, structures, arrays. These are initialized with values after every reset.
EEP_STOP_SEC_VAR_INIT_BOOLEAN	Variables	End of above section
EEP_START_SEC_VAR_INIT_32	Variables	Used for 32 bit variables, structures, arrays. These are initialized with values after every reset.
EEP_STOP_SEC_VAR_INIT_32	Variables	End of above section

Section name	Type of section	Description
EEP_START_SEC_VAR_INIT_UN↔ NSPECIFIED	Variables	Used for variables, structures, arrays, when the SIZE (alignment) does not fit the criterion of 8,16 or 32 bit. These variables are initialized with values after every reset.
EEP_STOP_SEC_VAR_INIT_UN↔ SPECIFIED	Variables	End of above section
EEP_START_SEC_VAR_CLEARE↔ D_BOOLEAN	Variables	Used for boolean variables, structures, arrays. These variables are cleared to zero by start-up code.
EEP_STOP_SEC_VAR_CLEARED↔ _BOOLEAN	Variables	End of above section
EEP_START_SEC_VAR_CLEARE↔ D_8	Variables	Used for variables, structures, arrays when the SIZE (alignment) fit the criteria of 8 bit. These variables are cleared to zero by start-up code.
EEP_STOP_SEC_VAR_CLEARED↔ _8	Variables	End of above section
EEP_START_SEC_VAR_CLEARE↔ D_8_NO_CACHEABLE	Variables	Used for variables, structures, arrays when the SIZE (alignment) fit the criteria of 8 bit. These variables are cleared to zero by start-up code.(no cacheable)
EEP_STOP_SEC_VAR_CLEARED↔ _8_NO_CACHEABLE	Variables	End of above section
EEP_START_SEC_VAR_CLEARE↔ D_16	Variables	Used for variables, structures, arrays when the SIZE (alignment) fit the criteria of 16 bit. These variables are cleared to zero by start-up code.
EEP_STOP_SEC_VAR_CLEARED↔ _16	Variables	End of above section
EEP_START_SEC_VAR_CLEARE↔ D_32	Variables	Used for variables, structures, arrays when the SIZE (alignment) fit the criteria of 32 bit. These variables are cleared to zero by start-up code.
EEP_STOP_SEC_VAR_CLEARED↔ _32	Variables	End of above section
EEP_START_SEC_VAR_CLEARE↔ D_UNSPECIFIED_NO_CACHEABLE	Non-Cacheable Variables	Used for variables, structures, arrays when the SIZE (alignment) does not fit the criteria of 8,16 or 32 bit, and that have to be stored in a non-cacheable memory section. These variables are cleared to zero by start-up code.
EEP_STOP_SEC_VAR_CLEARED↔ _UNSPECIFIED_NO_CACHEABLE	Non-Cacheable Variables	End of above section

7.2 Linker command file

Memory shall be allocated for every section defined in the driver's "<Module>"_MemMap.h.

Chapter 8

Integration Steps

This section gives a brief overview of the steps needed for integrating this module:

1. Generate the required module configuration(s). For more details refer to section [Files Required for Compilation](#)
2. Allocate the proper memory sections in the driver's memory map header file ("`<Module>_MemMap.h`") and linker command file. For more details refer to section [Sections to be defined in `<Module>\_MemMap.h`](#)
3. Compile & build the module with all the dependent modules. For more details refer to section [Building the Driver](#)

Chapter 9

External assumptions for driver

The section presents requirements that must be complied with when integrating the EEP driver into the application.

External Assumption Req ID	External Assumption Text
EA_RTD_00071	If interrupts are locked, a centralized function pair to lock and unlock interrupts shall be used.
EA_RTD_00082	When caches are enabled and data buffers are allocated in cacheable memory regions the buffers involved in DMA transfer shall be aligned with both start and end to cache line size. Note: Rationale: This ensures that no other buffers/variables compete for the same cache lines.
EA_RTD_00106	Standalone IP configuration and HL configuration of the same driver shall be done in the same project
EA_RTD_00107	The integrator shall use the IP interface only for hardware resources that were configured for standalone IP usage. Note: The integrator shall not directly use the IP interface for hardware resources that were allocated to be used in HL context.
EA_RTD_00108	The integrator shall use the IP interface to build a CDD, therefore the BSWMD will not contain reference to the IP interface
EA_RTD_00113	When RTD drivers are integrated with AutosarOS and User mode support is enabled, the integrator shall assure that the definition and declaration of all RTD functions needed to be called as trusted functions follow the naming convention <code>Call<Function_Name>TRUSTED(parameter1,parameter2,...)</code> in Integration/User code. They need to be visible in Os.h for the driver to call them. They will call RTD <code><Function_Name>()</code> as trusted functions in OS specific manner.

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: nxp.com/SalesTermsandConditions.

NXP, the NXP logo, NXP SECURE CONNECTIONS FOR A SMARTER WORLD, COOLFLUX, EMBRACE, GREENCHIP, HITAG, I2C BUS, ICODE, JCOP, LIFE VIBES, MIFARE, MIFARE CLASSIC, MIFARE DESFire, MIFARE PLUS, MIFARE FLEX, MANTIS, MIFARE ULTRALIGHT, MIFARE4MOBILE, MIGLO, NTAG, ROADLINK, SMARTLX, SMARTMX, STARPLUG, TOPFET, TRENCHMOS, UCODE, Freescale, the Freescale logo, Altivec, C-5, CodeTEST, CodeWarrior, ColdFire, ColdFire+, C-Ware, the Energy Efficient Solutions logo, Kinetis, Layerscape, MagniV, mobileGT, PEG, PowerQUICC, Processor Expert, QorIQ, QorIQ Qonverge, Ready Play, SafeAssure, the SafeAssure logo, StarCore, Symphony, VortiQa, Vybrid, Airfast, BeeKit, BeeStack, CoreNet, Flexis, MXC, Platform in a Package, QUICC Engine, SMARTMOS, Tower, TurboLink, and UMEMS are trademarks of NXP B.V. All other product or service names are the property of their respective owners. ARM, AMBA, ARM Powered, Artisan, Cortex, Jazelle, Keil, SecurCore, Thumb, TrustZone, and Vision are registered trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. ARM7, ARM9, ARM11, big.LITTLE, CoreLink, CoreSight, DesignStart, Mali, mbed, NEON, POP, Sensinode, Socrates, ULINK and Versatile are trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. All rights reserved. Oracle and Java are registered trademarks of Oracle and/or its affiliates. The Power Architecture and Power.org word marks and the Power and Power.org logos and related marks are trademarks and service marks licensed by Power.org.

© 2023 NXP B.V.

